

TP 1

Régression polynomiale

L'objectif de cet exercice est d'appréhender la méthode des moindres carrés linéaires pour estimer les paramètres d'un modèle polynomial à partir de données expérimentales.

1.1 Construction du jeu de données

Pour évaluer les performances des méthodes numériques, il est d'usage de simuler des jeux de données via un modèle dont on connaît la «vraie»¹ valeur. Cette «vraie» valeur du vecteur des paramètres n'est bien sûr pas utilisée dans la procédure de résolution du problème posé.

1. Pour $\mathbf{x} = (0:0.5:15)'$; de dimension $N \times 1$, simuler le modèle polynomial

$$\mathbf{y} = h_0 + h_1\mathbf{x} + h_2\mathbf{x}^2 + h_3\mathbf{x}^3 + h_4\mathbf{x}^4$$

pour un vecteur des paramètres vais, de dimension $p \times 1$, donné par

$$\mathbf{h} = [500; 10; -10; -5; 0.4];$$

Représenter $\mathbf{y}(\mathbf{x})$ à l'aide de la commande `plot`.

Solution. Pensez à toujours commenter votre code, et à donner des noms aux paramètres dans vos scripts, de manière à ce que votre code soit lisible et flexible. Evitez les accents dans vos commentaires.

```
% TP1: Regression polynomiale

%% 1.1 Construction du jeu de données
x = (0:0.5:15)';           % grille de discrétisation (vecteur colonne)
N = length(x);             % taille de la grille

h = [500; 10; -10; -5; 0.4]; % coefficients du polynome

y = h(1)*x.^0 + h(2)*x + ... % vecteur des mesures
    h(3)*x.^2 + h(4)*x.^3 + ...
    h(5)*x.^4;

% affichage
clf, plot(x,y,'-.','linewidth',2,'markersize',10);
legend('Echantillons non bruités (inconnus)');
```

1. La notion de valeur «vraie» des paramètres n'a aucun sens en pratique car cela supposerait qu'il y a un «vrai» modèle; or un modèle mathématique, aussi fin soit-il, n'est qu'une approximation de la réalité qui est trop complexe pour être parfaitement modélisée.

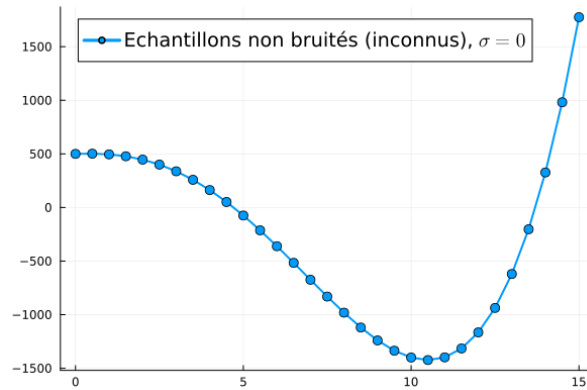


FIGURE 1.1 – Données non bruitées (gauche) et bruit seul (droite)

2. Pour s'approcher de la réalité, les incertitudes (souvent qualifiées de bruit) sur les mesures sont simulées par une suite stationnaire² de N variables aléatoires distribuées selon une loi normale $\mathcal{N}(\mu, \sigma^2)$ de moyenne $\mu = 0$ et de variance $\sigma^2 = 10^2$: `b = mu + sigma*randn(N,1);`. Ces incertitudes $\mathbf{b}$ sont supposées additives et les observations disponibles sont notées

$$\mathbf{y}_b = \mathbf{y} + \mathbf{b}.$$

Représenter les signaux $\mathbf{y}_b$ et $\mathbf{b}$ et relancer plusieurs fois le code. Que constatez-vous ?

Solution.

```
mu = 0;           % moyenne du bruit
sigma = 10;       % ecart-type du bruit
b = mu + sigma*randn(N,1); % bruit gaussien centre

yb = y + b;      % signal bruité

% affichage
clf;
subplot(1,2,1), plot(x,y,'-.','linewidth',2,'markersize',10);
hold on, plot(x,yb,'-x','linewidth',2,'markersize',10);
%
subplot(1,2,2), plot(x,b,'linewidth',2);
```

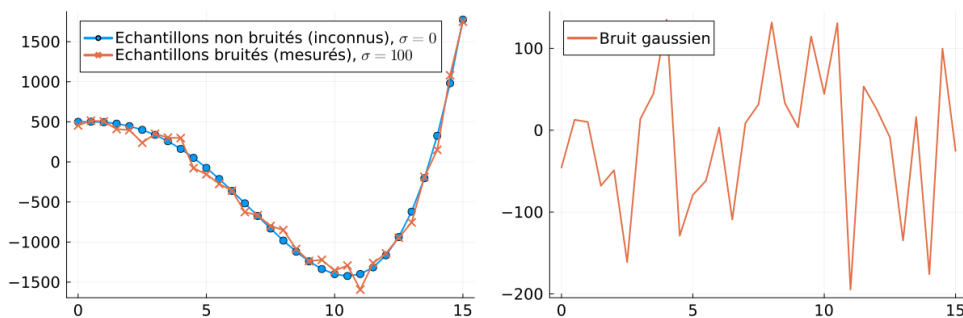


FIGURE 1.2 – Données bruitées

Afin de relancer plusieurs fois ce code facilement, on peut l'insérer dans une boucle `for`. Afin de simplement visualiser le résultat à chaque itération, on peut par exemple indiquer

2. L'hypothèse de stationnarité traduit le fait que l'on suppose que moyenne et variance des incertitudes ne varient pas au cours du temps. Il s'agit ici d'une suite de variables aléatoires indépendantes et indétachement distribuées, cette terminologie est souvent abrégée par i.i.d.. Lorsque la moyenne est nulle, l'indépendance des variables aléatoires confère à la densité spectrale du bruit une propriété particulière et l'on parle alors de bruit blanc.

à Matlab `drawnow` (pour qu'il dessine bien chaque fois), et `pause(0.2)`, pour qu'il fasse une pause de 0.2 secondes à chaque itération.

```
mu = 0;           % moyenne du bruit
sigma = 100;      % ecart-type du bruit

n_tries = 10;
for i=1:n_tries
    b = mu + sigma*randn(N,1);      % bruit gaussien centre
    yb = y + b;                    % signal bruité

    % affichage
    clf;
    subplot(1,2,1), plot(x,y,'-.','linewidth',2,'markersize',10);
    hold on, plot(x,yb,'-x','linewidth',2,'markersize',10);
    %
    subplot(1,2,2), plot(x,b,'linewidth',2);

    drawnow;
    pause(0.2);
end
```

On constate que les observations bruitées sont bien sûr différentes à chaque fois : on modélise bien un aléa sur nos mesures. Afin d'observer visuellement l'effet du bruit sur les graphiques, j'ai choisi $\sigma = 100$ dans les tests.

3. Calculer l'énergie du bruit W_b puis modifier la variance de celui-ci. Que constatez-vous ?

Solution. Traçons la valeur de W_b en fonction de l'écart-type σ du bruit, pour une série de valeurs de σ (en échelles logarithmiques).

```
n_scales = 4;
Wb = zeros(n_scales,1); % vecteur dans lequel on stockera les valeurs
for scale = 0:(n_scales-1)
    sigma = 10^scale; % ne pas donner le meme nom qu'avant
    b_test = mu + sigma*randn(N,1); % idem

    Wb(scale) = 1/N * norm(b_test)^2; % energie du bruit (stockee)
end

loglog(10.^(0:(n_scales-1)), Wb, 'linewidth', 2); % echelle log en x et en y
```

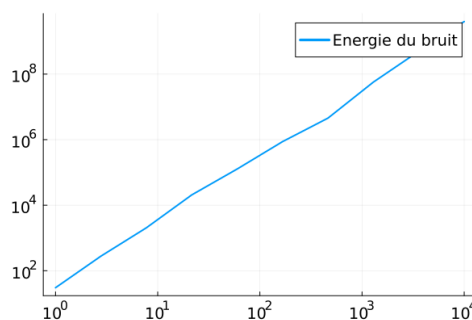


FIGURE 1.3 – Energie du bruit en fonction de σ

On constate que l'énergie du bruit croît linéairement avec σ .

1.2 Moindres carrées linéaires

1. Construire la matrice de Vandermonde $\mathbf{X}$ à partir du vecteur $\mathbf{x}$.

Solution. La matrice de Vandermonde $\mathbf{X}$ est simplement la matrice du système liant $\mathbf{y}$ et $\mathbf{h}$:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^4 \\ 1 & x_2 & x_2^2 & \dots & x_2^4 \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^4 \end{bmatrix}$$

En Matlab, cela donne :

```
%1.2 Moindres carres lineaires
X = [x(:).^0, x(:), x(:).^2, x(:).^3, x(:).^4]; % matrice de Vandermonde
```

La commande `(:)` permet de s'assurer que le vecteur est sous forme colonne. On vérifiera facilement que $\mathbf{y} = \mathbf{X}\mathbf{h}$.

2. Déterminer le vecteur $\hat{\mathbf{h}}$ en utilisant la formulation matricielle des moindres carrés et comparer le résultat obtenu avec la valeur «vraie» des paramètres. L'instruction `inv` permet d'inverser une matrice.

Solution. La solution des moindres carrés s'obtient via la pseudo-inverse de la matrice $\mathbf{X}$ du système. Lorsque la matrice est de rang colonne plein (c'est-à-dire lorsque le rang de la matrice est égal à son nombre de colonnes, ce qui est très souvent le cas pour des matrices avec peu de colonnes et beaucoup de lignes), la pseudo-inverse est donnée par la formule

$$\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top.$$

Il ne reste plus qu'à appliquer cette formule en Matlab.

```
% pseudo-inverse avec la fonction inv
X_pinv = inv(X'*X)*X';

% estimateur des moindres carres
h_est1 = X_pinv * y;
```

3. Dans le cas général, il n'est pas recommandé d'utiliser la programmation via `inv` qui est coûteuse en termes de calcul et peut souffrir de problèmes de conditionnement. Plusieurs alternatives sont alors possibles :

```
% *****
% moindres carres via élimination gaussienne
h_est2 = (X'*X)\X'*y_b;

% *****
% moindres carres via factorisation QR
[Q,R] = qr(X);
QTY = Q'*y_b;
opts_UT.UT = true;
h_est3 = linsolve(R,QTY,opts_UT);
Cond_QR = cond(R);

% *****
% moindres carres via SVD
[U,S,V] = svd(X,'econ');
h_est4 = V*inv(S)*U'*y_b;
Cond_SVD = cond(S);

% *****
% moindres carres via pseudo-inverse (obtenue apres SVD)
h_est5 = pinv(X)*y_b;
```

Tester ces différentes alternatives. En principe, les résultats devraient être identiques.

Dans le cas général il convient d'utiliser les approches fondées sur une factorisation QR ou sur une décomposition en valeurs singulières SVD. Si la SVD est plus complexe que la décomposition QR elle fournit le conditionnement du problème et permet la régularisation de problèmes mal conditionnés.

- Afficher la matrice S des valeurs singulières et vérifier que le rapport des valeurs singulières extrêmes donne le conditionnement au sens de la norme 2.

Solution. La matrice étant diagonale, on affiche ses valeurs diagonales non nulles.

```
% Affichage des valeurs singulières
s_vals = diag(S);
semilogy(s_vals(s_vals>0), '-.', 'linewidth', 2, 'markersize', 10);
```

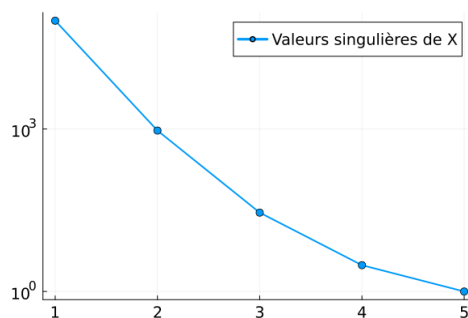


FIGURE 1.4 – Valeurs singulières (non nulles)

On constate que la matrice est mal conditionnée : le rapport $\sigma_{\max}/\sigma_{\min}$ est de l'ordre de 10^5 . On vérifie facilement que ce ratio est égal au conditionnement de la matrice pour la norme 2 (fonction `cond` de Matlab).

```
cond1 = max(s_vals)/min(s_vals(s>0));
cond2 = cond(X);

fprintf('Le rapport des valeurs singulieres extremes est: %.6d\n Le
        conditionnement obtenu via la fonction cond est: %.6d\n\n', cond1, cond2);
```

En pratique, cela signifie que la solution estimées par les moindres carrés (sans régularisation, donc à partir de la seule pseudo-inverse) sera instable en présence de bruit.

- Calculer et représenter la sortie estimée $\hat{\mathbf{y}}(x)$ ainsi que les résidus $\varepsilon(x) = \mathbf{y}_b(x) - \hat{\mathbf{y}}(x)$.

Solution. La sortie estimée se déduit immédiatement à partir des coefficients estimés `h_est` (avec la méthode que vous préférez) et de la matrice X .

```
% Sortie estimee
y_est = X*h_est;

# Affichage
clf;
subplot(1,2,1), plot(x,y, '-.', 'linewidth', 2, 'markersize', 10);
hold on;
plot(x,yb, '-x', 'linewidth', 2, 'markersize', 10);
plot(x,y_est, '-o', 'linewidth', 2, 'makersize', 10);
legend('Vrais échantillons', 'Echantillons bruités', 'Reconstruction');

subplot(1,2,2), plot(x,yb-y_est, 'linewidth', 2);
legend('Résidus');
```

- Que vaut le minimum du critère d'optimisation ? Comparer cette valeur à l'énergie du bruit additif calculée précédemment W_b .

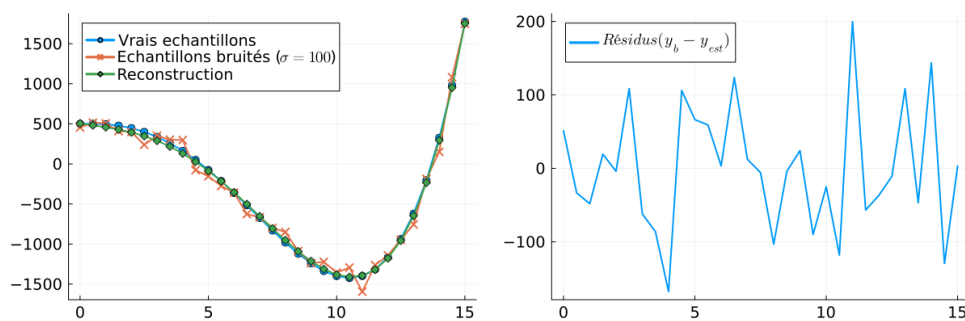


FIGURE 1.5 – Signal reconstruit (gauche) et résidu (droite)

Solution.

7. Tester également le cas où il n'y a pas de bruit sur les données.

Solution. 1.3 Sélection de l'ordre du polynôme

Lorsque l'on cherche à ajuster un polynôme de degré d à des données expérimentales, l'ordre de ce polynôme est généralement inconnu. Se pose alors le problème de sélection de l'ordre le plus approprié qui peut se formuler comme un problème d'optimisation discrète : rechercher le meilleur ordre au sens d'un certain critère.

- (a) Considérons dans un premier temps que le critère retenu soit celui des moindres carrés. Calculer les valeurs de ce critère pour d variant de 1 à 10. Que constatez-vous ? A partir de quelle valeur de d ce critère pourra-t-il être nul ?

Solution. On constaterait que le critère diminue avec d . Lorsque $d = N + 1$ où N est le nombre de points, on est sûr que l'erreur des moindres carrés sera à zéro, car il existe toujours un polynôme de degré $(N + 1)$ qui interpole les N points (le polynôme interpolateur de Lagrange). Vous pourrez facilement vous convaincre de ce résultat en essayant pour des petites valeurs de N (1 ou 2). Le critère des moindres carrés seul n'empêche pas le phénomène de *surapprentissage*, qui correspond au cas où le modèle colle trop aux données et n'est plus généralisable : ici par exemple, un modèle qui interpole toutes les données bruitées sera un très mauvais estimateur du polynôme ayant servi à générer les données (de degré 4). Pour remédier à cela, il faut soit trouver des heuristiques sur le modèle (comme le critère d'Akaike ci-dessous), soit recourir à de la régularisation (cf TP 2).

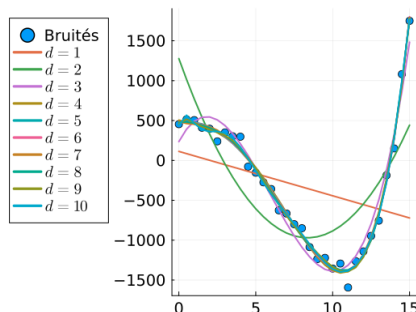


FIGURE 1.6 – Reconstruction pour différents degrés de polynôme

- (b) Afin de palier l'inconvénient illustré à la question précédente, les techniques de sélection d'ordre s'appuient sur des critères qui pénalisent le nombre de paramètres p afin de privilégier des modèles parcimonieux (pour cet exercice $p = d + 1$). En effet,

l'incertitude d'estimation qui pèse sur chaque paramètre s'accroît avec le nombre de paramètres (l'information disponible via les données, qui est fixe, se répartit sur un plus grand nombre d'entre eux).

Le critère d'information d'Akaike (AIC) permet de réaliser cette sélection d'ordre en retenant celui qui minimise

$$AIC = 2 \ln(\|\varepsilon\|_2^2) + 2p.$$

Déterminer l'ordre du polynôme qui minimise le critère d'information d'Akaike.

Solution. On calcule le critère d'Akaike tel que défini dans le sujet, et on trace ses valeurs en fonction du degré d pour d allant de 1 (approximation linéaire) à 10 (approximation par un polynôme de degré 10).

```
% Evaluation du critere d'Akaike
dmax = 10;
akaike = zeros(dmax,1);
for d=1:dmax
    Xd = x.(0:d); % matrice de Vandermonde d'ordre d
    hd = pinv(Xd)*yb; % coefficients estimes par moindres carres
    yd = Xd*hd; % signal reconstruit avec le modele d'ordre d

    res = yb-yd; % residus a l'ordre d

    akaike(d) = 2*log(norm(res)^2) + 2*(d+1);
end

plot(1:dmax,akaike,'linewidth',2);
legend('Critère d Akaike');
xlabel('Degré du polynôme (modèle)');
```

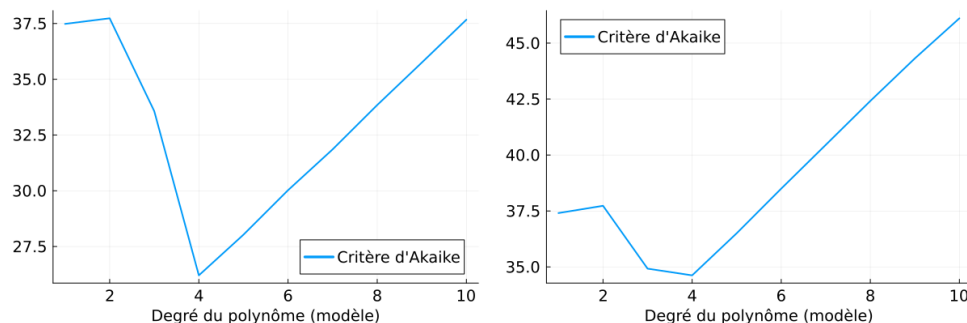


FIGURE 1.7 – Critère d'Akaike, dans le cas $\sigma = 10$ (gauche) et $\sigma = 100$ (droite)

Je vous montre les résultats pour deux intensités de bruit. On voit que lorsque le bruit est trop grand, le critère d'Akaike ne retrouvera pas toujours la bonne valeur du degré (4 dans notre cas), comme on l'observe sur la figure de droite (où le valeur du degré donnant le plus petit critère est 3).

TP 2

Déconvolution d'un signal triangulaire

Le problème considéré est celui de la reconstruction d'un signal x , supposé inconnu car non accessible à une mesure directe, ayant subi un filtrage gaussien inhérent à l'instrument de mesure. Il s'agit d'un filtrage linéaire d'usage courant en traitement d'image ; il est utilisé, ici, dans un contexte monodimensionnel. La réponse impulsionnelle (RI) d'un filtre gaussien est définie par :

$$h(t) = \frac{1}{\sigma_h \sqrt{2\pi}} \exp\left(-\frac{(t - m_h)^2}{2\sigma_h^2}\right) \quad (2.1)$$

Le signal y mesuré en sortie du filtre, seul accessible, est supposé être corrompu par des incertitudes additives. En effet, les mesures en sortie de filtre sont à minima sujettes au bruit de quantification dû au convertisseur analogique-numérique de la chaîne de mesure.

Il s'agit d'appréhender le caractère instable de l'opération de déconvolution, lorsque celle-ci est effectuée de façon « naïve », soit par division polynomiale (fonction `deconv` de Matlab), soit par filtrage inverse (dans le domaine fréquentiel ou par moindres carrés qui est l'équivalent matriciel). En présence d'erreurs (de bruit) sur les données, même très faibles, les résultats fournis par ces méthodes d'inversion peuvent, en effet, être inexploitable. On parle problème inverse *mal posé*. La réponse en fréquence du système à inverser permet de caractériser le degré de difficulté du problème. L'exercice illustrera la (les) solution(s) obtenues par régularisation linéaire quadratique.

2.1 Construction du jeu de données

La fréquence d'échantillonnage est fixée à la valeur $fe = 1$ Hz. Les échantillons du signal d'entrée sont notés $x(n)$, ceux de la réponse impulsionnelle du filtre $h(n)$ et ceux du signal mesuré en sortie $y(n)$. Le signal mesuré en sortie est sujet à des incertitudes additives notées $b(n)$ et le problème direct peut alors être modélisé comme suit :

$$y(n) = (x * h)(n) + b(n). \quad (2.2)$$

Simulation du signal d'entrée. Le signal d'entrée, supposé être triangulaire, est défini par

$$x(t) = \begin{cases} \frac{t}{L_x/2}, & \forall t \in [0, \frac{L_x}{2}] \\ \frac{-t + L_x}{L_x/2}, & \forall t \in [\frac{L_x}{2}, L_x]. \end{cases}$$

Simuler le signal à temps discret $x(n)$ correspondant, pour une durée d'observation $L_x = 100$ s.

Aide : remplacer t par nT_e dans l'équation définissant $x(t)$. Il est possible d'utiliser une instruction du type $\mathbf{x} = [\mathbf{x1}, \mathbf{x2}]$, où $\mathbf{x1}$ et $\mathbf{x2}$ représentent deux tronçons successifs du signal. De manière générale, il est conseillé d'adapter le script ci-dessous qui permet de maîtriser les différents paramètres de la discrétisation.

```
Lx = 100;           % duree d'observation du signal d'entree (secondes)
fe = 2;            % frequence d'echantillonnage (Herz)
Te = 1/fe;         % periode d'echantillonnage (secondes)
Nx = Lx/Te;        % nombre d'echantillons du signal
nx = 0:Nx-1;       % indice temporel (entier)
tx = nx*Te;         % base de temps (secondes)
```

Simulation du filtre. Pour les besoins de la simulation, déterminer la réponse impulsionnelle $h(n)$ du filtre gaussien obtenue par discrétisation de l'équation (??). Le filtre est de plus supposé causal et ses paramètres seront pris égaux respectivement à $m_h = 15$ et $\sigma_h = 4$. Enfin, la réponse impulsionnelle est tronquée à un horizon de 30 s. Représenter cette réponse impulsionnelle.

Rappel : la réponse impulsionnelle d'un filtre causal est nul aux temps négatifs

Simulation du problème direct, première approche : fonction conv. Comme le filtre gaussien est un filtre **linéaire invariant**, il est possible de calculer sa sortie par le produit de convolution discret donné par l'équation (??).

1. Simuler le signal de sortie non bruité $\mathbf{y}_{nb}(n)$ à l'aide de la fonction `conv`.
2. Si le signal de sortie $\mathbf{y}$ est mesuré à l'aide d'un convertisseur analogique-numérique, il est sujet au bruit de quantification. L'opération de quantification peut être simulée par l'instruction `y=round(ynb*210)/210`. Simuler le signal quantifié $\mathbf{y}$ et représenter celui-ci à l'aide de la fonction `stairs`.
3. Déterminer et représenter le bruit de quantification $b(n)$.

Simulation du problème direct, deuxième approche : formulation matricielle. Dans cette partie, le signal de sortie non bruité est simulé par l'équation matricielle $\mathbf{y}_{nb} = \mathbf{H}\mathbf{x}$. L'usage d'un paramétrage soigné est recommandé pour permettre de modifier aisément les conditions de simulation et d'affichage des résultats.

1. Déterminer la dimension N_y du vecteur de sortie $\mathbf{y}_{nb}$ à partir des dimensions N_x et N_h des vecteurs d'entrée $\mathbf{x}$ et de réponse impulsionnelle $\mathbf{h}$ afin d'obtenir le même nombre d'échantillons que par la fonction `conv`.
2. Construire la matrice de convolution $\mathbf{H}$ à l'aide de la fonction `toeplitz`. Pour cela, il est proposé d'initialiser le premier vecteur ligne et le premier vecteur colonne de $\mathbf{H}$ avec des vecteurs nuls de dimension appropriée à l'aide la fonction `zeros`, puis de remplacer certains de leurs éléments avec les éléments de $\mathbf{h}$.
3. Déterminer la sortie non bruitée par une opération matricielle, puis appliquer la fonction permettant de simuler l'erreur de troncature.

2.2 Déconvolution

2.2.1 Déconvolution «naïve»et constat de l'instabilité

1. Reconstruire le signal d'entrée en utilisant l'opération de déconvolution Matlab (`deconv`), en se plaçant, dans un premier temps, dans le cas idéal où l'on a accès à la sortie non bruitée $\mathbf{y}_{nb}$.

2. À présent, reconstruire le signal d'entrée à partir des échantillons mesurés du signal de sortie $\mathbf{y}$ (cadre réaliste). Le signal reconstruit sera appelé $\mathbf{x}_{rec}$. Que vaut le RSB du signal de sortie ?
3. Étudier de façon empirique l'influence de la «forme» de la réponse impulsionnelle sur la qualité de la reconstruction en modifiant la valeur du paramètre σ_h .

2.2.2 Interprétation fréquentielle de l'inversion

2.2.3 Déconvolution par moindres carrés (ℓ_2)

1. Calculer le signal déconvolué à l'aide de l'estimateur des moindres carrés.
2. Tracer le signal déconvolué et calculer l'énergie de l'erreur de reconstruction.
3. Tracer le spectre des valeurs singulières de la matrice $\mathbf{H}^\top \mathbf{H}$ et déterminer son conditionnement.
4. Comment le conditionnement évolue-t-il en fonction de la valeur du paramètre σ_j du filtre ?

2.2.4 Déconvolution par régularisation linéaire quadratique ($\ell_2\ell_2$)

Cette partie concerne la mise en œuvre de la solution obtenue par minimisation d'un critère composite de fonctionnelles L et R quadratiques : $J(\mathbf{x}) = L(\mathbf{x}) + \lambda R(\mathbf{x})$.

1. Construire la matrice carrée $\mathbf{D}_1$, de dimension N_x , telle que le produit $\mathbf{D}_1 \mathbf{x}$ donne les différences premières¹ : $x(n+1)x(n)$.

Aide : On s'aidera du code suivant

```
dcol1 = zeros(1:Nx);
dcol1(1:2) = [1 -1];
dlig1 = zeros(1,Nx);
dlig1(1,1) = dcol(1,1);
D1 = toeplitz(dcol1,dlig1);
```

Variante : remplacer la seconde ligne du script ci-dessus par

```
dcol1(1:3) = [1 -2 1];
```

donne accès aux différences secondes définissant alors la matrice $\mathbf{D}_2$.

2. Calculer la solution régularisée en déterminant le coefficient de régularisation λ de manière empirique².
3. En simulation, le signal à reconstruire est parfaitement connu, il est donc possible de déterminer la valeur du coefficient de régularisation qui minimise l'énergie de l'erreur de reconstruction. Tracer l'évolution de l'énergie de l'erreur de reconstruction en fonction de λ . Ecrire une procédure permettant d'approcher la valeur optimale de λ , par recherche exhaustive sur un intervalle, à l'aide de la fonction `min`.
4. Tracer la «courbe en L» en représentant, sur une échelle log-log (fonction `loglog`), la fonctionnelle de régularisation $R(\hat{\mathbf{x}}(\lambda)) = \|\mathbf{D}_1 \hat{\mathbf{x}}(\lambda)\|_2^2$ en fonction du critère d'adéquation aux données $L(\hat{\mathbf{x}}(\lambda)) = \|\mathbf{y} - \mathbf{H} \hat{\mathbf{x}}(\lambda)\|_2^2$ (moindres carrés).

Aide : vous pouvez faire varier λ de la manière suivante

1. Les différences premières, lorsqu'elles sont divisées par les pas T_e , correspondent à une approximation de la dérivée première.
 2. Par exemple, en testant plusieurs valeurs dans un intervalle conséquent, comme $[10^{-7}, 10^2]$.

```
expmin = -7;
expmax = 2;
pas = 0.1;
id = 0;
for p = expmin:pas:expmax
    lambda = 10^p;
    id = id+1;
    ...
```

5. Que se passe-t-il lorsque l'on remplace la matrice $\mathbf{D}_1$ par la matrice identité I_{N_x} ou par $\mathbf{D}_2$? Quelle pénalisation constitue une façon appropriée de coder les connaissances a priori dont on dispose sur le signal à reconstruire.